

Team Note of BunCua Team

MinhhHoangPersia - Pain - Lingg

Compiled on September 18, 2026

Contents

1 Template

2 Data Structure

2.1	Segment Tree Walk	1
2.2	Persistent Segment Tree	1
2.3	Trie	2
2.4	Persistent Trie	2
2.5	Fenwick Tree	3
2.6	Fenwick Tree 2D	3
2.7	XOR Basis	4
2.8	DSU Rollback	5
2.9	Ordered Set	5
2.10	Line Container	6
2.11	Deque CHT	7

3 Graphs

3.1	2 SAT	7
3.2	LCA $O(1)$	8
3.3	Centroid Decomposition	8
3.4	Heavy Light Decomposition	8
3.5	Bellman Ford	9
3.6	Floyd	9
3.7	Dinic	9
3.8	Min Cost Max Flow	10
3.9	Dsu On Tree(Sack)	11
3.10	MO On Tree	12
3.11	Knapsack On Tree	14

4 DP

4.1	DP Digit Optimization 1	14
4.2	DP Digit Optimization 2	15
4.3	DP SoS	15
4.4	DP Divide And Conquer	15

5 Strings

5.1	KMP	16
5.2	Z Function	16
5.3	Suffix Array	16
5.4	Aho Corasick	17

6 Math

6.1	Extended Euclid	18
6.2	CRT	19
6.3	Lucas Theorem	19
6.4	Euler Totient	20
6.5	Mobius Function	20
6.6	Miller Rabin	21

7 Geometry

8 Miscellaneous

8.1	Alien Trick	22
8.2	Interval Container	22
8.3	Ternary Search	23

1 Template

2 Data Structure

2.1 Segment Tree Walk

```
/* Find first element in range [u, v] have value >= val */
int findPos(int id, int l, int r, int u, int v, ll val) {
    if(l > v || r < u) return n + 1;
    if(st[id] < val) return n + 1;
    if(l == r) return l;
    push(id);
```

```
    int mid = (l + r) / 2;
    int res = findPos(id * 2, l, mid, u, v, val);
    if(res == n + 1) {
        res = findPos(id * 2 + 1, mid + 1, r, u, v, val);
    }
    return res;
}
```

2.2 Persistent Segment Tree

```
const int N = 3e5 + 5;
const int LOG = 20;
const int MAX_NODES = N * 40;

struct Node {
    int left, right, v;
    Node(int _left = 0, int _right = 0, int _v = 0) {
        left = _left;
        right = _right;
        v = _v;
    }
} st[MAX_NODES];

int cnt = 0;
int ver[N], nver = 0;

void refine(int id) {
    st[id].v = st[st[id].left].v + st[st[id].right].v;
}

int build(int l, int r) {
    if (l == r) {
        cnt++;
        st[cnt] = Node(0, 0, 0);
        return cnt;
    }
    int mid = (l + r) >> 1;
    int cur = ++cnt;
    st[cur].left = build(l, mid);
    st[cur].right = build(mid + 1, r);
    refine(cur);
    return cur;
}

int update(int id, int l, int r, int u, int x) {
    if (l == r) {
        cnt++;
        st[cnt] = Node(0, 0, st[id].v + x);
        return cnt;
    }
    int mid = (l + r) >> 1;
    int cur = ++cnt;
    if (u <= mid) {
        st[cur].left = update(st[id].left, l, mid, u, x);
        st[cur].right = st[id].right;
    } else {
        st[cur].left = st[id].left;
        st[cur].right = update(st[id].right, mid + 1, r, u, x);
    }
    refine(cur);
    return cur;
}

int get(int id, int l, int r, int u, int v) {
    if (l > v || r < u) return 0;
    if (l >= u && r <= v) return st[id].v;
    int mid = (l + r) >> 1;
    return get(st[id].left, l, mid, u, v) + get(st[id].right,
        mid + 1, r, u, v);
}
```

```

void add_version(int l, int r, int u, int x) {
    nver++;
    ver[nver] = update(ver[nver - 1], l, r, u, x);
}
/*
    USAGE EXAMPLE (Solving Distinct Values in Range - DQUERY):

    // 1. Initialize base tree
    cnt = nver = 0;
    ver[0] = build(1, n);

    // 2. Build versions tracking the latest occurrence of each
    value
    for (int i = 1; i <= n; i++) {
        // If value appeared before, remove its previous position
        if (mark[a[i]] > 0) {
            ver[nver] = update(ver[nver], 1, n, mark[a[i]], -1);
        }

        // Add current position
        nver++;
        ver[nver] = update(ver[nver - 1], 1, n, i, 1);

        iver[i] = ver[nver]; // iver[i] stores the version for
        prefix i
        mark[a[i]] = i;
    }

    // 3. Query distinct values in [L, R]
    int ans = get(iver[R], 1, n, L, R);
*/

```

2.3 Trie

```

const int LOG_MAX = 29;
const int MAX_NODES = 3e5 * 32;

struct Node {
    int c[2];
    int cnt;

    Node() {
        c[0] = c[1] = 0;
        cnt = 0;
    }
};

Node trie[MAX_NODES];
int node_cnt = 0;

void init_trie() {
    for (int i = 0; i <= node_cnt; i++) {
        trie[i] = Node();
    }
    node_cnt = 0;
}

void insert(int val) {
    int cur = 0;
    for (int i = LOG_MAX; i >= 0; i--) {
        int x = (val >> i) & 1;
        if (trie[cur].c[x] == 0) {
            trie[cur].c[x] = ++node_cnt;
        }
        cur = trie[cur].c[x];
        trie[cur].cnt++;
    }
}

bool search(int val) {
    int cur = 0;
    for (int i = LOG_MAX; i >= 0; i--) {
        int x = (val >> i) & 1;
        cur = trie[cur].c[x];
        if (cur == 0 || trie[cur].cnt == 0) {
            return false;
        }
    }
    return trie[cur].cnt > 0;
}

```

```

int count_occurrences(int val) {
    int cur = 0;
    for (int i = LOG_MAX; i >= 0; i--) {
        int x = (val >> i) & 1;
        cur = trie[cur].c[x];
        if (cur == 0 || trie[cur].cnt == 0) {
            return 0;
        }
    }
    return trie[cur].cnt;
}

void erase(int val) {
    if (!search(val)) return;
    int cur = 0;
    for (int i = LOG_MAX; i >= 0; i--) {
        int x = (val >> i) & 1;
        cur = trie[cur].c[x];
        trie[cur].cnt--;
    }
}

void update(int old_val, int new_val) {
    if (!search(old_val)) return;
    erase(old_val);
    insert(new_val);
}

```

2.4 Persistent Trie

```

const int LOG_MAX = 29;
const int N = 1e5 + 5;
const int MAX_NODES = N * 32;

int t[MAX_NODES][2];
int num[MAX_NODES];
int root[N];
int cnt = 0;

void init_trie() {
    for (int i = 0; i <= cnt; i++) {
        t[i][0] = t[i][1] = num[i] = 0;
    }
    cnt = 0;
    root[0] = 0;
}

int newnode() {
    cnt++;
    t[cnt][0] = t[cnt][1] = num[cnt] = 0;
    return cnt;
}

int add_version(int prev_root, int val) {
    int cur_root = newnode();
    int cur = cur_root;
    int old = prev_root;
    for (int i = LOG_MAX; i >= 0; i--) {
        int x = (val >> i) & 1;
        t[cur][0] = t[old][0];
        t[cur][1] = t[old][1];
        t[cur][x] = newnode();
        cur = t[cur][x];
        old = t[old][x];
        t[cur][0] = t[old][0];
        t[cur][1] = t[old][1];
        num[cur] = num[old] + 1;
    }
    return cur_root;
}

// 4. Query Maximum XOR with 'val' using combinations of
// versions.
// Example: Query on a tree path from u to v with LCA p,
// parent of p is pp.
// The effective count of a node is: num[x] + num[y] - num[p]
// - num[pp].
int query_max_xor(int rootX, int rootY, int rootP, int rootPP,
int val) {
    int res = 0;

```

```

int x = rootX, y = rootY, p = rootP, pp = rootPP;
for (int i = LOG_MAX; i >= 0; i--) {
    int v = (val >> i) & 1;
    int count_opposite = num[t[x][v ^ 1]] + num[t[y][v ^ 1]]
        - num[t[p][v ^ 1]] - num[t[pp][v ^ 1]];
    if (count_opposite > 0) {
        res |= (1 << i);
        x = t[x][v ^ 1];
        y = t[y][v ^ 1];
        p = t[p][v ^ 1];
        pp = t[pp][v ^ 1];
    } else {
        x = t[x][v];
        y = t[y][v];
        p = t[p][v];
        pp = t[pp][v];
    }
}
return res;
}

/*
USAGE EXAMPLE (Max XOR in an array from L to R):

// 1. Build prefix versions
init_trie();
for (int i = 1; i <= n; i++) {
    root[i] = add_version(root[i - 1], a[i]);
}

// 2. Query Max XOR with 'val' in subarray [L, R]
// For arrays, the effective tree is root[R] - root[L-1].
// You can use a modified query function that only takes 2
roots:
// count_opposite = num[t[rootR][v ^ 1]] -
num[t[rootL_minus_1][v ^ 1]];
*/

```

2.5 Fenwick Tree

```

struct Fenwick {
    int n;
    vector<long long> bit;

    Fenwick(int _n) {
        n = _n;
        bit.assign(n + 1, 0);
    }

    void update(int idx, long long delta) {
        for (; idx <= n; idx += idx & -idx) {
            bit[idx] += delta;
        }
    }

    long long query(int idx) {
        long long sum = 0;
        for (; idx > 0; idx -= idx & -idx) {
            sum += bit[idx];
        }
        return sum;
    }

    long long query(int l, int r) {
        if (l > r) return 0;
        return query(r) - query(l - 1);
    }

    // O(log N) Bit Jumping: Finds the FIRST index where prefix
    sum >= val.
    // Note: This only works if all elements in the BIT are
    non-negative!
    int lower_bound(long long val) {
        if (val <= 0) return 1;
        int pos = 0;
        long long sum = 0;
        for (int i = __lg(n); i >= 0; i--) {
            int next_pos = pos + (1 << i);
            if (next_pos <= n && sum + bit[next_pos] < val) {
                sum += bit[next_pos];
                pos = next_pos;
            }
        }
    }
}

```

```

    }
}
// pos is the largest index such that prefix sum < val
// So pos + 1 is the first index such that prefix sum >=
val
return pos + 1;
}
};

```

2.6 Fenwick Tree 2D

```

/*
Idea:
- 1-based indexing for a grid of size N x M.
- When we add `v` to a 2D difference array at (i, j), it
contributes to
the prefix sum S(X, Y) (where X >= i and Y >= j) by:
v * (X - i + 1) * (Y - j + 1)
= v * [ (X+1)(Y+1) - (X+1)*j - (Y+1)*i + i*j ]
- To maintain this, we need 4 2D BITs to store the sum of:
1. v
2. v * i
3. v * j
4. v * i * j
*/

struct Fenwick2D {
    int n, m;
    vector<vector<long long>> bit1, bit2, bit3, bit4;

    Fenwick2D(int _n, int _m) {
        n = _n;
        m = _m;
        bit1.assign(n + 2, vector<long long>(m + 2, 0));
        bit2.assign(n + 2, vector<long long>(m + 2, 0));
        bit3.assign(n + 2, vector<long long>(m + 2, 0));
        bit4.assign(n + 2, vector<long long>(m + 2, 0));
    }

    void add(int x, int y, long long val) {
        for (int i = x; i <= n; i += i & -i) {
            for (int j = y; j <= m; j += j & -j) {
                bit1[i][j] += val;
                bit2[i][j] += val * x;
                bit3[i][j] += val * y;
                bit4[i][j] += val * x * y;
            }
        }
    }

    void update(int x1, int y1, int x2, int y2, long long val) {
        add(x1, y1, val);
        add(x2 + 1, y1, -val);
        add(x1, y2 + 1, -val);
        add(x2 + 1, y2 + 1, val);
    }

    // Internal Prefix Query: gets the sum of submatrix from (1,
    1) to (x, y)
    long long query_prefix(int x, int y) {
        long long res = 0;
        for (int i = x; i > 0; i -= i & -i) {
            for (int j = y; j > 0; j -= j & -j) {
                res += bit1[i][j] * (x + 1) * (y + 1)
                    - bit2[i][j] * (y + 1)
                    - bit3[i][j] * (x + 1)
                    + bit4[i][j];
            }
        }
        return res;
    }

    // Range Query: gets the sum of submatrix from (x1, y1) to
    (x2, y2)
    long long query(int x1, int y1, int x2, int y2) {
        return query_prefix(x2, y2)
            - query_prefix(x1 - 1, y2)
            - query_prefix(x2, y1 - 1)
            + query_prefix(x1 - 1, y1 - 1);
    }
};

```

2.7 XOR Basis

```

/*
XOR BASIS (Linear Basis over GF(2))

Idea:
- Represents a set of numbers using a minimal set of
  linearly independent
  numbers (basis).
- Any XOR sum of a subset in the original set can be formed
  by a XOR sum
  of a subset of the basis.
- The size of the basis never exceeds the number of bits
  (LOG_D).

Complexity:
- Insert / Check / Get Max: O(LOG_D)
- Merge two bases: O(LOG_D^2)
- Build RREF: O(LOG_D^2)
- K-th Smallest/Largest: O(LOG_D)
*/
const int LOG_D = 60; // 60 for values up to 10^18, 30 for
values up to 10^9

struct XorBasis {
    long long basis[LOG_D];
    int sz; // Number of elements currently in the basis
    vector<long long> reduced_basis; // Stores the canonical
    form for K-th queries

    XorBasis() {
        memset(basis, 0, sizeof(basis));
        sz = 0;
    }

    // 1. Insert a value. Returns true if successfully inserted
    // (i.e., the value is linearly independent from the current
    basis).
    bool insert(long long x) {
        for (int i = LOG_D - 1; i >= 0; i--) {
            if ((x >> i) & 1) {
                if (!basis[i]) {
                    basis[i] = x;
                    sz++;
                    return true;
                }
                x ^= basis[i];
            }
        }
        return false; // x became 0, meaning it was already
        spanned by the basis
    }

    // 2. Check if a value can be formed by XORing a subset of
    the basis
    bool check(long long x) {
        for (int i = LOG_D - 1; i >= 0; i--) {
            if ((x >> i) & 1) {
                if (!basis[i]) return false;
                x ^= basis[i];
            }
        }
        return true;
    }

    // 3. Get the maximum possible XOR sum using the basis.
    long long get_max(long long start_val = 0) {
        long long res = start_val;
        for (int i = LOG_D - 1; i >= 0; i--) {
            if ((res ^ basis[i]) > res) {
                res ^= basis[i];
            }
        }
        return res;
    }

    // 4. Merge this basis with another basis
    void merge(const XorBasis& other) {
        for (int i = 0; i < LOG_D; i++) {
            if (other.basis[i]) {
                insert(other.basis[i]);
            }
        }
    }

    // K-TH LARGEST / SMALLEST QUERIES
    // Note: These functions only consider DISTINCT values
    generated by the basis.
    // There are exactly 2^sz distinct values (including 0 from
    the empty subset).

    // 5. Build Reduced Row Echelon Form (Canonical Form)
    // MUST be called after all inserts and BEFORE querying k-th
    values!
    void build() {
        reduced_basis.clear();
        for (int i = LOG_D - 1; i >= 0; i--) {
            for (int j = i - 1; j >= 0; j--) {
                // If the j-th bit of basis[i] is set, use basis[j] to
                turn it off
                if ((basis[i] >> j) & 1) {
                    basis[i] ^= basis[j];
                }
            }
        }
        // Collect non-zero elements into the reduced_basis array
        (in ascending order)
        for (int i = 0; i < LOG_D; i++) {
            if (basis[i]) {
                reduced_basis.push_back(basis[i]);
            }
        }
    }

    // 6. Get the k-th smallest distinct XOR sum (1-indexed).
    // k = 1 returns 0 (the empty subset).
    // Returns -1 if k is out of bounds (k > 2^sz).
    long long get_kth_smallest(long long k) {
        long long total_combinations = 1LL << sz;
        if (k < 1 || k > total_combinations) return -1;

        long long rank = k - 1; // 0-indexed rank
        long long res = 0;
        for (int i = 0; i < sz; i++) {
            if ((rank >> i) & 1) {
                res ^= reduced_basis[i];
            }
        }
        return res;
    }

    // 7. Get the k-th largest distinct XOR sum (1-indexed).
    // k = 1 returns the absolute maximum possible XOR sum.
    // Returns -1 if k is out of bounds (k > 2^sz).
    long long get_kth_largest(long long k) {
        long long total_combinations = 1LL << sz;
        if (k < 1 || k > total_combinations) return -1;

        // The k-th largest is equivalent to the
        (total_combinations - k)-th smallest
        long long rank = total_combinations - k;
        long long res = 0;
        for (int i = 0; i < sz; i++) {
            if ((rank >> i) & 1) {
                res ^= reduced_basis[i];
            }
        }
        return res;
    }
};

/*
USAGE EXAMPLE:

signed main() {
    int n = 3;
    long long a[] = {2, 4, 6}; // 2 (010), 4 (100), 6 (110)

    XorBasis xb;
    for (int i = 0; i < n; i++) xb.insert(a[i]);

    // 1. Build the canonical form before calling k-th
    queries

```

```

    xb.build();

    // The distinct XOR sums generated are: 0, 2, 4, 6
    // Since 6 is just  $2^2$ , the basis size (sz) will be 2.
    // ( $2^2 = 4$  combinations)

    cout << "Max XOR sum: " << xb.get_max() << "\n"; // 6

    cout << "1st smallest: " << xb.get_kth_smallest(1) <<
    "\n"; // 0
    cout << "2nd smallest: " << xb.get_kth_smallest(2) <<
    "\n"; // 2

    cout << "1st largest: " << xb.get_kth_largest(1) << "\n";
    // 6
    cout << "3rd largest: " << xb.get_kth_largest(3) << "\n";
    // 2

    return 0;
}
*/

```

2.8 DSU Rollback

```

struct DSU_Rollback {
    vector<int> parent, sz;
    int components;

    // Stores the history of unions as pairs {child, parent}
    vector<pair<int, int>> history;

    DSU_Rollback(int n) {
        parent.assign(n + 1, 0);
        sz.assign(n + 1, 1);
        components = n;
        for (int i = 1; i <= n; i++) {
            parent[i] = i;
        }

        // Find set WITHOUT path compression
        int find_set(int v) {
            while (v != parent[v]) {
                v = parent[v];
            }
            return v;
        }

        // Union two sets by size and record the operation
        bool union_sets(int a, int b) {
            a = find_set(a);
            b = find_set(b);
            if (a == b) return false;

            // Union by size: always attach the smaller tree (a) to
            // the larger one (b)
            if (sz[a] > sz[b]) {
                swap(a, b);
            }

            // Perform union
            parent[a] = b;
            sz[b] += sz[a];
            components--;

            // Record the operation: 'a' became a child of 'b'
            history.push_back({a, b});
            return true;
        }

        // Take a snapshot of the current state (returns the size of
        // the history stack)
        int snapshot() {
            return history.size();
        }

        // Rollback operations until the history stack size is equal
        // to `checkpoint`
        void rollback(int checkpoint) {
            while (history.size() > checkpoint) {
                int a = history.back().first;

```

```

                int b = history.back().second;
                history.pop_back();

                // Undo the union operation
                sz[b] -= sz[a];
                parent[a] = a;
                components++;
            }
        }
    };

    /*
    USAGE EXAMPLE (Dynamic Connectivity / Divide & Conquer on
    Time):

    signed main() {
        int n = 5;
        DSU_Rollback dsu(n);

        dsu.union_sets(1, 2);
        dsu.union_sets(3, 4);
        cout << "Components: " << dsu.components << "\n"; //
        Output: 3

        // 1. Save the current state
        int checkpoint = dsu.snapshot();

        // 2. Do some temporary unions
        dsu.union_sets(2, 3);
        dsu.union_sets(4, 5);
        cout << "Components after temp unions: " <<
        dsu.components << "\n"; // Output: 1

        // 3. Rollback to the saved state
        dsu.rollback(checkpoint);

        // The state is perfectly restored!
        cout << "Components after rollback: " << dsu.components
        << "\n"; // Output: 3

        return 0;
    }
    */

```

2.9 Ordered Set

```

/*
GNU PBDS ORDERED SET (Policy-Based Data Structure)

Idea:
- A balanced binary search tree (Red-Black Tree) with order
  statistics.
- Works exactly like std::set but with two additional  $O(\log N)$ 
  functions:
    1. find_by_order(k): returns an iterator to the k-th
       smallest element (0-indexed).
    2. order_of_key(x): returns the number of elements
       strictly smaller than x.

IMPORTANT FOR MULTISSET:
- Do NOT use `less_equal<T>` for multiset if you need to
  use `.erase()`.
  It will delete ALL occurrences of the value, or break the
  tree structure.
- The best and safest way to implement an ordered_multiset
  is to use
  `pair<T, int>` where the second element is a unique
  counter/ID.

*/
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>

using namespace std;
using namespace __gnu_pbds;

// 1. Ordered Set (Distinct elements only)
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;

```

```
// 2. Ordered Multiset (Allows duplicates safely by using
pair<T, int>)
template <typename T>
using ordered_multiset = tree<pair<T, int>, null_type,
less<pair<T, int>>, rb_tree_tag,
tree_order_statistics_node_update>;

/*
USAGE EXAMPLE:

signed main() {
    //
    -----
    // EXAMPLE 1: ORDERED SET (Distinct Elements)
    //
    -----
    ordered_set<int> os;
    os.insert(10);
    os.insert(30);
    os.insert(20);
    os.insert(30); // Ignored, as 30 is already present

    // find_by_order(k): Returns iterator to the k-th
    smallest element (0-indexed)
    cout << *os.find_by_order(0) << "\n"; // Output: 10
    cout << *os.find_by_order(2) << "\n"; // Output: 30

    // order_of_key(x): Returns the number of elements
    STRICTLY smaller than x
    cout << os.order_of_key(25) << "\n"; // Output: 2 (10
    and 20)
    cout << os.order_of_key(20) << "\n"; // Output: 1 (only
    10)

    os.erase(20); // Erases 20 normally

    //
    -----
    // EXAMPLE 2: ORDERED MULTISSET (With Duplicates)
    //
    -----
    ordered_multiset<int> oms;
    int timer = 0; // Unique ID counter to distinguish
    duplicates

    // Insert: use pair {value, ++timer}
    oms.insert({10, ++timer});
    oms.insert({20, ++timer});
    oms.insert({20, ++timer});
    oms.insert({30, ++timer});

    // order_of_key: just pass {value, -1} to get count of
    elements < value
    cout << oms.order_of_key({20, -1}) << "\n"; // Output: 1
    (only {10})
    cout << oms.order_of_key({30, -1}) << "\n"; // Output: 3
    ({10}, {20}, {20})

    // find_by_order works exactly the same, but returns a
    pair
    cout << oms.find_by_order(1)->first << "\n"; // Output:
    20
    cout << oms.find_by_order(2)->first << "\n"; // Output:
    20

    // Erase one occurrence of 20:
    // Find its iterator using lower_bound with -1, then
    erase the iterator
    auto it = oms.lower_bound({20, -1});
    if (it != oms.end() && it->first == 20) {
        oms.erase(it);
    }

    return 0;
}
*/
```

2.10 Line Container

```
/*
```

```
DYNAMIC CONVEX HULL TRICK (LINE CONTAINER)
Idea:
- Maintains the upper envelope of a set of lines  $y = kx + m$ .
- Allows dynamically adding lines and querying the MAXIMUM
y value at
    a given x in  $O(\log N)$  time.
- For MINIMUM queries: insert(-k, -m) and negate the query
result: -query(x).
How it works (KACTL approach):
- Uses std::multiset to keep lines sorted by slope 'k'.
- Maintains the intersection point 'p' with the next line.
- Uses C++14 heterogeneous lookup (less<>) to binary search
by 'x' value
    using the 'p' attribute.
Complexity:
- Add line: Amortized  $O(\log N)$ 
- Query:  $O(\log N)$ 

*/
const long long LINF = 2e18;
struct Line {
    mutable long long k, m, p;
    bool operator<(const Line& o) const { return k < o.k; }
    bool operator<(long long x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
    long long div(long long a, long long b) {
        return a / b - ((a ^ b) < 0 && a % b);
    }

    bool isect(iterator x, iterator y) {
        if (y == end()) { x->p = LINF; return false; }
        if (x->k == y->k) {
            x->p = x->m > y->m ? LINF : -LINF;
        } else {
            x->p = div(y->m - x->m, x->k - y->k);
        }
        return x->p >= y->p;
    }

    // Adds a line  $y = kx + m$  to the envelope
    void add(long long k, long long m) {
        auto z = insert({k, m, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() && (--x)->p >= y->p) {
            isect(x, erase(y));
        }
    }

    // Queries the maximum y value at a given x
    long long query(long long x) {
        assert(!empty());
        auto l = *lower_bound(x);
        return l.k * x + l.m;
    }
};

/*
USAGE EXAMPLE:
signed main() {
    LineContainer lc;
    //
    -----
    // EXAMPLE 1: MAXIMUM QUERIES (Default behavior)
    //
    -----
    // Add lines:  $y = 2x + 3$ ,  $y = 1x + 10$ ,  $y = 3x - 5$ 
    lc.add(2, 3);
    lc.add(1, 10);
    lc.add(3, -5);

    cout << "Max at x=2: " << lc.query(2) << "\n"; // y =
    1*(2) + 10 = 12
    cout << "Max at x=10: " << lc.query(10) << "\n"; // y =
    3*(10) - 5 = 25

    //
    -----
    // EXAMPLE 2: MINIMUM QUERIES (Trick)
```

```
//
-----
LineContainer min_lc;

// We want to find MINIMUM of: y = 2x + 3, y = 1x + 10, y
= 3x - 5
// Trick: Insert (-k, -m)
min_lc.add(-2, -3);
min_lc.add(-1, -10);
min_lc.add(-3, 5); // -(-5) = 5

// Trick: Negate the query result
cout << "Min at x=2: " << -min_lc.query(2) << "\n"; // y
= 2*(2) + 3 = 7
cout << "Min at x=10: " << -min_lc.query(10) << "\n"; //
y = 1*(10) + 10 = 20

return 0;
}
*/
```

2.11 Deque CHT

```
/*
Trick for MINIMUM QUERIES:
- Add lines as: add(-k, -m)
- Query as: -query_increasing_x(x) or
-query_binary_search(x)
*/
struct Line {
    long long k, m;
    long long get(long long x) { return k * x + m; }
};

struct DequeCHT {
    deque<Line> dq;

    long long floor_div(long long a, long long b) {
        return a / b - ((a ^ b) < 0 && a % b);
    }

    long long intersect(Line a, Line b) {
        return floor_div(a.m - b.m, b.k - a.k);
    }

    // Adds a line y = k*x + m.
    // IMPORTANT: Slopes (k) must be inserted in non-decreasing
    order!
    void add(long long k, long long m) {
        Line L = {k, m};

        if (!dq.empty() && dq.back().k == k) {
            if (m <= dq.back().m) return;
            dq.pop_back();
        }

        while (dq.size() >= 2) {
            Line l1 = dq[dq.size() - 2];
            Line l2 = dq.back();
            if (intersect(l1, l2) >= intersect(l2, L)) {
                dq.pop_back();
            } else {
                break;
            }
        }
        dq.push_back(L);
    }

    // 1. O(1) Query: ONLY use when query values 'x' are
    monotonically INCREASING!
    long long query_increasing_x(long long x) {
        assert(!dq.empty());
        while (dq.size() >= 2 && dq[0].get(x) <= dq[1].get(x)) {
            dq.pop_front();
        }
        return dq[0].get(x);
    }

    // 2. O(log N) Query: Use when 'x' values are RANDOM / NOT
    SORTED
    long long query_binary_search(long long x) {
```

```
assert(!dq.empty());
int l = 0, r = dq.size() - 2;
int ans = dq.size() - 1;

while (l <= r) {
    int mid = l + (r - l) / 2;
    if (intersect(dq[mid], dq[mid + 1]) < x) {
        l = mid + 1;
    } else {
        ans = mid;
        r = mid - 1;
    }
}
return dq[ans].get(x);
}
};
```

3 Graphs

3.1 2 SAT

```
/*
- For a boolean variable x, we create two nodes: x (1..n)
and NOT(x) (n+1..2n).
- A clause (x OR y) is equivalent to: (!x => y) and (!y =>
x).
- If x and NOT(x) are in the same Strongly Connected
Component (SCC),
it's a contradiction -> NO solution.
- We assign TRUE to x if its SCC is topologically sorted
AFTER NOT(x)'s SCC.
(Tarjan assigns SCC IDs in reverse topological order, so
col[x] < col[NOT(x)])
*/

const int N = 2e4 + 5;

int n;
vector<int> G[2 * N];
int num[2 * N], low[2 * N], cnt;
int col[2 * N], ct;
vector<int> stk;
int ans[N];

int NOT(int x) {
    return (x <= n ? x + n : x - n);
}

void addedge(int x, int y) {
    G[NOT(x)].push_back(y);
    G[NOT(y)].push_back(x);
}

void tarjan(int u) {
    num[u] = low[u] = ++cnt;
    stk.push_back(u);
    for (int v : G[u]) {
        if (!col[v]) {
            if (num[v]) {
                low[u] = min(low[u], num[v]);
            } else {
                tarjan(v);
                low[u] = min(low[u], low[v]);
            }
        }
    }
    if (num[u] == low[u]) {
        ct++;
        while (true) {
            int v = stk.back();
            stk.pop_back();
            col[v] = ct;
            if (v == u) break;
        }
    }
}

bool solve() {
    rep(i, 1, 2 * n) {
        if (!num[i]) tarjan(i);
    }
}
```

```

rep(i, 1, n) {
    if (col[i] == col[NOT(i)]) return false;
    ans[i] = (col[i] < col[NOT(i)]);
}
return true;
}

```

3.2 LCA O(1)

```

const int N = 1e5 + 5;
const int LOG = 18;

```

```

vector<int> G[N];
int tin[N], cnt, rn[N];
int euler[2 * N], ct, first[N];
int depth[N];
int rmq[LOG][2 * N];

```

```

void dfs(int u, int p = 0) {
    tin[u] = ++cnt;
    rn[cnt] = u;
    euler[++ct] = tin[u];
    first[tin[u]] = ct;
    for (int v : G[u]) {
        if (v == p) continue;
        depth[v] = depth[u] + 1;
        dfs(v, u);
        euler[++ct] = tin[u];
    }
}

```

```

void build_lca() {
    for (int i = 1; i <= ct; i++) {
        rmq[0][i] = euler[i];
    }
    for (int j = 1; j < LOG; j++) {
        for (int i = 1; i <= ct - (1 << (j - 1)); i++) {
            rmq[j][i] = min(rmq[j - 1][i], rmq[j - 1][i + (1 << (j - 1))]);
        }
    }
}

```

```

int lca(int x, int y) {
    x = first[tin[x]];
    y = first[tin[y]];
    if (x > y) swap(x, y);
    int k = 31 - __builtin_clz(y - x + 1);
    int min_tin = min(rmq[k][x], rmq[k][y - (1 << k) + 1]);
    return rn[min_tin];
}

```

3.3 Centroid Decomposition

```

const int N = 3e5 + 5;
vector<int> G[N];
int sz[N];
bool deleted[N];
int centroid_par[N];

```

```

void get_sz(int u, int par) {
    sz[u] = 1;
    for (int v : G[u]) {
        if (v != par && !deleted[v]) {
            get_sz(v, u);
            sz[u] += sz[v];
        }
    }
}

```

```

int get_centroid(int u, int par, int total_nodes) {
    for (int v : G[u]) {
        if (v != par && !deleted[v]) {
            if (sz[v] > total_nodes / 2) {
                return get_centroid(v, u, total_nodes);
            }
        }
    }
    return u;
}

```

```

void solve_centroid(int centroid) {
    // TODO: Add your logic here
}

```

```

void build_centroid(int u, int p = -1) {
    get_sz(u, -1);
    int centroid = get_centroid(u, -1, sz[u]);
    deleted[centroid] = true;
    centroid_par[centroid] = p;
    solve_centroid(centroid);
    for (int v : G[centroid]) {
        if (!deleted[v]) {
            build_centroid(v, centroid);
        }
    }
}

```

3.4 Heavy Light Decomposition

```

const int N = 2e5 + 5;
vector<int> G[N];
int sz[N], son[N], depth[N], pre[N];
int in[N], cnt, top[N];

```

```

void dfs1(int u, int par) {
    sz[u] = 1;
    son[u] = 0;
    pre[u] = par;
    for (int v : G[u]) {
        if (v != par) {
            depth[v] = depth[u] + 1;
            dfs1(v, u);
            sz[u] += sz[v];
            if (sz[v] > sz[son[u]]) {
                son[u] = v;
            }
        }
    }
}

```

```

void rebuild(int u, int topp) {
    in[u] = ++cnt;
    top[u] = topp;
    if (son[u]) {
        rebuild(son[u], topp);
    }
    for (int v : G[u]) {
        if (v != pre[u] && v != son[u]) {
            rebuild(v, v);
        }
    }
}

```

```

int query_path(int x, int y) {
    int res = 0;
    while (top[x] != top[y]) {
        if (depth[top[x]] < depth[top[y]]) {
            swap(x, y);
        }
        res = max(res, get(1, 1, n, in[top[x]], in[x]));
        x = pre[top[x]];
    }
    if (depth[x] > depth[y]) {
        swap(x, y);
    }
    res = max(res, get(1, 1, n, in[x], in[y]));
    return res;
}

```

```

void update_path(int x, int y, int val) {
    while (top[x] != top[y]) {
        if (depth[top[x]] < depth[top[y]]) {
            swap(x, y);
        }
        x = pre[top[x]];
    }
    if (depth[x] > depth[y]) {
        swap(x, y);
    }
}

```



```

/*
  USAGE EXAMPLE:
  dfs1(1, 1);
  cnt = 0;
  rebuild(1, 1);
  // for(int i = 1; i <= n; i++) update(1, 1, n, in[i],
  a[i]);
*/

```

3.5 Bellman Ford

```

const long long LINF = 1e18;

struct Edge {
  int u, v;
  long long w;
};

int n, m;
vector<Edge> edges;
long long dist[2005];
int parent[2005];

bool bellman_ford(int src) {
  for (int i = 1; i <= n; i++) {
    dist[i] = LINF;
    parent[i] = -1;
  }
  dist[src] = 0;
  for (int i = 1; i <= n - 1; i++) {
    bool updated = false;
    for (auto e : edges) {
      if (dist[e.u] < LINF && dist[e.u] + e.w < dist[e.v]) {
        dist[e.v] = dist[e.u] + e.w;
        parent[e.v] = e.u;
        updated = true;
      }
    }
    if (!updated) break;
  }
  bool has_negative_cycle = false;
  for (int i = 1; i <= n; i++) {
    for (auto e : edges) {
      if (dist[e.u] < LINF && dist[e.u] + e.w < dist[e.v]) {
        dist[e.v] = -LINF;
        has_negative_cycle = true;
      }
    }
  }
  return has_negative_cycle;
}

/*
  HOW TO EXTRACT A NEGATIVE CYCLE (If needed)
  If you only need to print ONE negative cycle (like CSSES
  Cycle Finding):
  1. Keep an array `parent`.
  2. Run Bellman-Ford for exactly N iterations (no need for
  Phase 2 above).
  3. On the N-th iteration, if an edge (u, v) is relaxed, `v`
  is either in
     the negative cycle or reachable from it.
  4. To find a node strictly inside the cycle, go back `N`
  times using `parent`:
     int x = v;
     for (int i = 1; i <= n; i++) x = parent[x];
  5. Now `x` is guaranteed to be in the cycle. Trace back
  using `parent[x]`
     until you reach `x` again, push nodes to a vector, and
     reverse it.
*/

```

3.6 Floyd

```

const long long LINF = 1e18;
const int N = 505;

long long dist[N][N];
int nxt[N][N];

```

```

void init(int n) {
  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
      dist[i][j] = LINF;
      nxt[i][j] = -1;
    }
    dist[i][i] = 0;
  }
}

void add_edge(int u, int v, long long w) {
  if (w < dist[u][v]) {
    dist[u][v] = w;
    nxt[u][v] = v;
  }
}

void floyd_warshall(int n) {
  for (int k = 1; k <= n; k++) {
    for (int i = 1; i <= n; i++) {
      for (int j = 1; j <= n; j++) {
        if (dist[i][k] < LINF && dist[k][j] < LINF) {
          if (dist[i][k] + dist[k][j] < dist[i][j]) {
            dist[i][j] = dist[i][k] + dist[k][j];
            nxt[i][j] = nxt[i][k];
          }
        }
      }
    }
  }

  // 4. (Optional) Propagate Negative Cycles
  // If node k is part of a negative cycle, any path going
  // through k
  // will also have a length of -Infinity.
  for (int k = 1; k <= n; k++) {
    if (dist[k][k] < 0) {
      for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
          if (dist[i][k] < LINF && dist[k][j] < LINF) {
            dist[i][j] = -LINF;
          }
        }
      }
    }
  }

  // 5. Path Reconstruction
  // Returns the path from u to v, or an empty vector if no path
  // exists
  vector<int> get_path(int u, int v) {
    if (dist[u][v] == LINF) return {};

    vector<int> path;
    for (int at = u; at != v; at = nxt[at][v]) {
      if (at == -1) return {};
      path.push_back(at);
    }
    path.push_back(v);
    return path;
  }
}

```

3.7 Dinic

```

/*
  Complexity:
  - General Graph:  $O(V^2 * E)$ 
  - Unit Network (e.g., Bipartite Matching, weight = 1):  $O(E * \sqrt{V})$ 
  - Extremely fast in practice, much faster than
    Edmonds-Karp.
*/
const long long LINF = 1e18;

struct Edge {
  int u, v;
  long long cap, flow;
};

struct Dinic {

```

```

int n;
vector<Edge> edges;
vector<vector<int>> adj;
vector<int> level;
vector<int> ptr;

Dinic(int _n) {
    n = _n;
    adj.assign(n + 1, vector<int>());
    level.assign(n + 1, -1);
    ptr.assign(n + 1, 0);
}

void add_edge(int u, int v, long long cap, bool is_directed = true) {
    adj[u].push_back(edges.size());
    edges.push_back({u, v, cap, 0});

    adj[v].push_back(edges.size());
    edges.push_back({v, u, is_directed ? 0 : cap, 0});
}

bool bfs(int s, int t) {
    fill(level.begin(), level.end(), -1);
    level[s] = 0;
    queue<int> q;
    q.push(s);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int id : adj[u]) {
            int v = edges[id].v;
            if (level[v] == -1 && edges[id].cap - edges[id].flow > 0) {
                level[v] = level[u] + 1;
                q.push(v);
            }
        }
    }
    return level[t] != -1;
}

long long dfs(int u, int t, long long pushed) {
    if (pushed == 0 || u == t) return pushed;

    for (int& cid = ptr[u]; cid < adj[u].size(); ++cid) {
        int id = adj[u][cid];
        int v = edges[id].v;
        long long tr = edges[id].cap - edges[id].flow;

        if (level[u] + 1 != level[v] || tr == 0) continue;

        long long push = dfs(v, t, min(pushed, tr));
        if (push == 0) continue;

        edges[id].flow += push;
        edges[id ^ 1].flow -= push;
        return push;
    }
    return 0;
}

// Main function to calculate Max Flow
long long max_flow(int s, int t) {
    long long flow = 0;
    while (bfs(s, t)) {
        fill(ptr.begin(), ptr.end(), 0);

        while (long long pushed = dfs(s, t, LINF)) {
            flow += pushed;
        }
    }
    return flow;
}

// Min Cut Extraction: Nodes reachable from Source in the residual graph
vector<bool> get_min_cut(int s) {
    vector<bool> visited(n + 1, false);
    queue<int> q;
    q.push(s);

```

```

        visited[s] = true;

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (int id : adj[u]) {
                int v = edges[id].v;
                if (!visited[v] && edges[id].cap - edges[id].flow > 0) {
                    visited[v] = true;
                    q.push(v);
                }
            }
        }
        return visited; // visited[i] = true means 'i' is in the Source set
    }

    // Tracing: Print edges that carry flow
    void print_flow_paths() {
        cout << "Edges with positive flow:\n";
        for (int i = 0; i < edges.size(); i += 2) { // Check forward edges only
            if (edges[i].flow > 0) {
                cout << edges[i].u << " -> " << edges[i].v
                    << " : " << edges[i].flow << " / " << edges[i].cap << "\n";
            }
        }
    }
};
/*
USAGE EXAMPLE:
signed main() {
    int n = 4; // 4 vertices
    Dinic dinic(n);
    // add_edge(u, v, capacity)
    dinic.add_edge(1, 2, 3);
    dinic.add_edge(1, 3, 2);
    dinic.add_edge(2, 3, 1);
    dinic.add_edge(2, 4, 2);
    dinic.add_edge(3, 4, 4);
    int source = 1, sink = 4;
    cout << "Max Flow: " << dinic.max_flow(source, sink) << "\n"; // Output: 5
    return 0;
}
*/

```

3.8 Min Cost Max Flow

```

/*
- Uses Dijkstra instead of SPFA to find the cheapest augmenting path.
- Maintains a potential function `pi` for each node to handle negative weight residual edges.
- Edge weights are reweighted:  $w'(u, v) = w(u, v) + pi[u] - pi[v] \geq 0$ .
- Complexity:  $O(F * E \log V)$  where  $F$  is the flow. Fast and safe!
*/
const long long LINF = 1e18;

struct MCMFEdge {
    int u, v;
    long long cap, flow, cost;
};

struct MinCostMaxFlow {
    int n;
    vector<MCMFEdge> edges;
    vector<vector<int>> adj;
    vector<long long> pi, dist;
    vector<int> parent_edge;

    MinCostMaxFlow(int _n) {
        n = _n;
        adj.assign(n + 1, vector<int>());
        pi.assign(n + 1, 0);
        dist.assign(n + 1, 0);
    }

```

```

    parent_edge.assign(n + 1, -1);
}

void add_edge(int u, int v, long long cap, long long cost,
bool is_directed = true) {
    adj[u].push_back(edges.size());
    edges.push_back({u, v, cap, 0, cost});

    adj[v].push_back(edges.size());
    edges.push_back({v, u, is_directed ? 0 : cap, 0, -cost});
}

// Run SPFA once to set initial potentials.
// ONLY UNCOMMENT inside mcmf() if the original graph has
NEGATIVE edge costs.
void set_pi(int s) {
    fill(pi.begin(), pi.end(), LINF);
    vector<bool> in_queue(n + 1, false);
    queue<int> q;

    pi[s] = 0;
    q.push(s);
    in_queue[s] = true;

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        in_queue[u] = false;

        for (int id : adj[u]) {
            int v = edges[id].v;
            if (edges[id].cap - edges[id].flow > 0 && pi[v] >
pi[u] + edges[id].cost) {
                pi[v] = pi[u] + edges[id].cost;
                if (!in_queue[v]) {
                    q.push(v);
                    in_queue[v] = true;
                }
            }
        }
    }
}

// Dijkstra using reduced costs
bool dijkstra(int s, int t) {
    fill(dist.begin(), dist.end(), LINF);
    fill(parent_edge.begin(), parent_edge.end(), -1);

    priority_queue<pair<long long, int>, vector<pair<long
long, int>>, greater<pair<long long, int>>> pq;

    dist[s] = 0;
    pq.push({0, s});

    while (!pq.empty()) {
        long long d = pq.top().first;
        int u = pq.top().second;
        pq.pop();

        if (d > dist[u]) continue;

        for (int id : adj[u]) {
            int v = edges[id].v;
            long long remain_cap = edges[id].cap - edges[id].flow;

            if (remain_cap > 0) {
                long long reduced_cost = edges[id].cost + pi[u] -
pi[v];
                if (dist[v] > dist[u] + reduced_cost) {
                    dist[v] = dist[u] + reduced_cost;
                    parent_edge[v] = id;
                    pq.push({dist[v], v});
                }
            }
        }
    }
    return dist[t] != LINF;
}

pair<long long, long long> mcmf(int s, int t, long long
max_required_flow = LINF) {

```

```

    long long total_flow = 0;
    long long total_cost = 0;

    // Uncomment this line ONLY IF the initial graph contains
    negative costs
    // set_pi(s);

    while (total_flow < max_required_flow && dijkstra(s, t)) {
        long long push = max_required_flow - total_flow;
        int cur = t;
        while (cur != s) {
            int id = parent_edge[cur];
            push = min(push, edges[id].cap - edges[id].flow);
            cur = edges[id].u;
        }

        cur = t;
        while (cur != s) {
            int id = parent_edge[cur];
            edges[id].flow += push;
            edges[id ^ 1].flow -= push;
            total_cost += push * edges[id].cost;
            cur = edges[id].u;
        }

        total_flow += push;

        for (int i = 1; i <= n; i++) {
            if (dist[i] != LINF) {
                pi[i] += dist[i];
            }
        }
    }

    return {total_flow, total_cost};
}

// PATH TRACING: Prints all edges that are actively carrying
flow
void print_flow_paths() {
    cout << "Edges with positive flow (Min Cost Max Flow):\n";
    for (int i = 0; i < edges.size(); i += 2) {
        if (edges[i].flow > 0) {
            cout << edges[i].u << " -> " << edges[i].v
<< " | Flow: " << edges[i].flow << " / " <<
edges[i].cap
<< " | Unit Cost: " << edges[i].cost
<< " | Total Edge Cost: " << edges[i].flow *
edges[i].cost << "\n";
        }
    }
}
};

```

3.9 Dsu On Tree(Sack)

```

/*
- Efficiently answers OFFLINE queries about subtrees in  $O(N \log N)$  time.
- Solves problems like: "For each node U, how many nodes in
its subtree
    have a specific property?"
Algorithm:
1. Find the "heavy" child (largest subtree) for each node.
2. DFS traversal:
    - Traverse all "light" children, answer their queries,
then CLEAR their data.
    - Traverse the "heavy" child, answer its queries, and
KEEP its data.
    - Add the data of the light children and the current
node itself.
    - Answer queries for the current node.
    - If the current node is a light child (`keep ==
false`), CLEAR all its data.
*/

const int N = 2e5 + 5;
vector<int> G[N];

// Variables for Euler tour and Heavy-Light logic
int in[N], out[N], rn[N], timer;

```

```

int sz[N], big[N];

// 1. First DFS: Precompute subtree sizes, Euler tour, and heavy children
void dfs_sz(int u, int par) {
    in[u] = ++timer;
    rn[timer] = u;
    sz[u] = 1;
    big[u] = 0;

    for (int v : G[u]) {
        if (v != par) {
            dfs_sz(v, u);
            sz[u] += sz[v];
            if (sz[big[u]] < sz[v]) {
                big[u] = v; // Update heavy child
            }
        }
    }
    out[u] = timer;
}

// Struct to manage the global state (Modify this according to the problem!)
// Example: Counting frequencies of values
struct GlobalState {
    int freq[N];
    int count_of_freq[N];

    void add(int node) {
        // int val = a[node];
        // count_of_freq[++freq[val]]++;
    }

    void remove(int node) {
        // int val = a[node];
        // count_of_freq[freq[val]--]--;
    }

    // int get_answer(...) { return ...; }
} state;

// 2. Second DFS: DSU on Tree (Sack)
void dsu_on_tree(int u, int par, bool keep) {
    // Step 2.1: Traverse light children and DO NOT keep their data
    for (int v : G[u]) {
        if (v != par && v != big[u]) {
            dsu_on_tree(v, u, false);
        }
    }

    // Step 2.2: Traverse the heavy child and KEEP its data
    if (big[u] != 0) {
        dsu_on_tree(big[u], u, true);
    }

    // Step 2.3: Add light children's data and the current node's data
    for (int v : G[u]) {
        if (v != par && v != big[u]) {
            for (int i = in[v]; i <= out[v]; i++) {
                state.add(rn[i]);
            }
        }
    }
    state.add(u); // Add the current node itself

    // Step 2.4: ANSWER QUERIES for node `u` here
    // for (auto query : queries[u]) {
    //     ans[query.id] = state.get_answer(query.k);
    // }

    // Step 2.5: If we are a light child, clear our data so it doesn't affect
    // other branches of our parent.
    if (!keep) {
        for (int i = in[u]; i <= out[u]; i++) {
            state.remove(rn[i]);
        }
    }
}

```

```

}

/*
USAGE EXAMPLE:
signed main() {
    // ... Read graph and queries ...
    // ... Save queries as `queries[u].push_back({k, query_id})` ...
    timer = 0;
    dfs_sz(1, -1);
    // Start DSU on tree from the root, keeping is irrelevant for the root
    dsu_on_tree(1, -1, false);
    // Print answers
    // ...
    return 0;
}
*/

```

3.10 MO On Tree

```

/*
MO'S ALGORITHM ON TREES (WITH UPDATES - 3D MO)
Idea:
- Flattens the tree into an array using Euler Tour. Each node appears exactly TWICE: when we enter it (tin) and when we leave it (tout).
- A node is considered "in the current path" if it appears exactly ONCE in our active range [L, R]. We maintain a boolean array `f[u]` to toggle the presence of node `u`.

Query Translation (Path from U to V, assuming tin[U] < tin[V]):
- Case 1: U is an ancestor of V.
    Query range: [tin[U], tin[V]]. LCA is already included.
- Case 2: U is NOT an ancestor of V.
    Query range: [tout[U], tin[V]]. LCA is NOT in this range, so we must manually add/remove the LCA during the query.

Complexity:
- Without updates (2D Mo): Block size = N / sqrt(Q), Time = O(N * sqrt(Q))
- With updates (3D Mo): Block size = N^(2/3), Time = O(N^(5/3))
*/
const int N = 2e5 + 5;
// For 3D Mo (with updates), BLOCK size should be roughly N^(2/3).
// For N = 100,000, N^(2/3) is approx 2150.
const int BLOCK = 2150;

int n, q;
int a[N];
vector<int> G[N];

// Euler Tour & LCA
int tin[N], tout[N], timer;
int tour[N * 2]; // Needs to be 2 * N because each node appears twice
int up[18][N], depth[N];

// Mo's State
int freq[N], distinct_count;
int f[N]; // f[x] = 1 if node x is currently in the path, 0 otherwise
int ans[N];

struct Query {
    int l, r, lca_idx, t, id;

    bool operator<(const Query& other) const {
        int bl = l / BLOCK;
        int br = r / BLOCK;
        int obl = other.l / BLOCK;
        int obr = other.r / BLOCK;
        if (bl != obl) return bl < obl;
    }
}

```

```

        if (br != obr) return (bl & 1) ? br < obr : br > obr;
        return (br & 1) ? t < other.t : t > other.t;
    }
};

struct Update {
    int x, new_val, old_val;
};

vector<Query> queries;
vector<Update> updates;

// 1. Build Euler Tour and LCA table
void dfs(int u, int par) {
    up[0][u] = par;
    for (int i = 1; i < 18; i++) {
        if (up[i - 1][u] != -1) {
            up[i][u] = up[i - 1][up[i - 1][u]];
        } else {
            up[i][u] = -1;
        }
    }

    tin[u] = ++timer;
    tour[timer] = u;

    for (int v : G[u]) {
        if (v != par) {
            depth[v] = depth[u] + 1;
            dfs(v, u);
        }
    }

    tout[u] = ++timer;
    tour[timer] = u;
}

int get_lca(int u, int v) {
    if (depth[u] < depth[v]) swap(u, v);
    int delta = depth[u] - depth[v];
    for (int i = 0; i < 18; i++) {
        if ((delta >> i) & 1) u = up[i][u];
    }
    if (u == v) return u;
    for (int i = 17; i >= 0; i--) {
        if (up[i][u] != up[i][v]) {
            u = up[i][u];
            v = up[i][v];
        }
    }
    return up[0][u];
}

// 2. State Manipulation Functions
void add(int val) {
    if (++freq[val] == 1) distinct_count++;
}

void sub(int val) {
    if (freq[val]-- == 1) distinct_count--;
}

// Toggle a node: if it's already in the path, remove it.
// Otherwise, add it.
void toggle(int idx) {
    int x = tour[idx];
    f[x] ^= 1;
    if (f[x]) add(a[x]);
    else sub(a[x]);
}

// Apply point update dynamically
void apply_update(int x, int val, int old_val) {
    if (f[x]) {
        add(val);
        sub(old_val);
    }
    a[x] = val;
}

/*
    USAGE EXAMPLE:

```

```

signed main() {
    // ... Read N, Q, array `a`, and tree edges ...

    timer = 0;
    memset(up, -1, sizeof(up));
    depth[1] = 0;
    dfs(1, -1);

    // Note: If values are large, do Coordinate Compression
    // on `a` and updates first!

    // ... Reading Queries and Updates ...
    int time_counter = 0;
    for (int i = 1; i <= q; i++) {
        int type; cin >> type;
        if (type == 1) { // Path Query
            int u, v; cin >> u >> v;
            if (tin[u] > tin[v]) swap(u, v); // Ensure tin[u] <=
            tin[v]

            int lca = get_lca(u, v);
            if (lca == u) {
                // u is ancestor of v -> LCA is already inside
                [tin[u], tin[v]]
                queries.push_back({tin[u], tin[v], -1,
                    time_counter, (int)queries.size() + 1});
            } else {
                // u is NOT ancestor of v -> LCA is missing, need
                to add it manually
                queries.push_back({tout[u], tin[v], tin[lca],
                    time_counter, (int)queries.size() + 1});
            }
        } else { // Point Update
            int x, val; cin >> x >> val;
            time_counter++;
            updates.push_back({x, val, a[x]});
            a[x] = val; // temporarily apply to get `old_val` for
            future queries
        }
    }

    // Rollback the array `a` to its initial state before
    processing Mo
    for (int i = (int)updates.size() - 1; i >= 0; i--) {
        a[updates[i].x] = updates[i].old_val;
    }

    sort(queries.begin(), queries.end());

    int L = 1, R = 0, T = 0;
    for (const auto& q : queries) {
        while (R < q.r) toggle(++R);
        while (L > q.l) toggle(--L);
        while (R > q.r) toggle(R--);
        while (L < q.l) toggle(L++);

        while (T < q.t) {
            apply_update(updates[T].x, updates[T].new_val,
                updates[T].old_val);
            T++;
        }
        while (T > q.t) {
            T--;
            apply_update(updates[T].x, updates[T].old_val,
                updates[T].new_val);
        }

        // Manually toggle LCA if needed
        if (q.lca_idx != -1) toggle(q.lca_idx);

        ans[q.id] = distinct_count;

        // Untoggle LCA to restore state
        if (q.lca_idx != -1) toggle(q.lca_idx);
    }

    for (int i = 1; i <= queries.size(); i++) {
        cout << ans[i] << "\n";
    }
    return 0;
}

```

```

    }
    */

```

3.11 Knapsack On Tree

```

/*
DP KNAPSACK ON TREE (Prerequisite DP / Connected Component
Knapsack)
Idea:
- Problem: Choose a subset of nodes to maximize total value
such that the
    total weight <= W.
    CONSTRAINT: If a node is chosen, its parent MUST also be
    chosen.
    (e.g., Course selection with prerequisites, or picking a
    connected
    component rooted at node 1).
- Naive DP merges subtrees taking  $O(N * W^2)$ .
OPTIMIZATION (DFS Flattening Trick):
- We flatten the tree using DFS Pre-Order traversal.
- For a node at pre-order index `i`, its entire subtree
spans contiguously
    from index `i` to `i + sz[u] - 1`.
- Transitions:
    1. Skip node `i`: We CANNOT pick any node in its subtree.
    We jump
        directly to index `i + sz[u]`.
    2. Take node `i`: We pay `weight[u]`, gain `value[u]`,
    and move to
        index `i + 1` (allowing us to pick its children).
Complexity:
- Time:  $O(N * W)$ 
- Space:  $O(N * W)$ 
*/
struct TreeKnapsack {
    int n;
    vector<vector<int>> adj;
    vector<long long> weight, value;

    // DFS state
    vector<int> sz;
    vector<int> rev_dfn; // rev_dfn[i] = node ID at DFS
    pre-order index `i`
    int timer;

    TreeKnapsack(int _n) {
        n = _n;
        adj.assign(n + 1, vector<int>());
        weight.assign(n + 1, 0);
        value.assign(n + 1, 0);
        sz.assign(n + 1, 0);
        rev_dfn.assign(n + 1, 0);
        timer = 0;
    }

    // Add directed edge from parent to child
    void add_edge(int u, int v) {
        adj[u].push_back(v);
    }

    // Build the Pre-Order array and calculate subtree sizes
    void dfs(int u) {
        timer++;
        rev_dfn[timer] = u;
        sz[u] = 1;

        for (int v : adj[u]) {
            dfs(v);
            sz[u] += sz[v];
        }
    }

    // Solves the knapsack problem for a specific weight limit
    `W`
    long long solve(int root, int W) {
        timer = 0;
        dfs(root); // Flatten the tree

        // dp[i][w] = max value considering nodes in pre-order
        from index `i` to `n`,
        // having exactly `w` weight capacity available.

```

```

// We use size (n + 2) to safely handle the jump to `n +
1` when skipping the last nodes.
vector<vector<long long>> dp(n + 2, vector<long long>(W +
1, 0));

// Iterate backwards through the pre-order indices
for (int i = n; i >= 1; i--) {
    int u = rev_dfn[i];
    int jump = i + sz[u]; // The index to jump to if we skip
    subtree of `u`

    for (int w = 0; w <= W; w++) {
        // Option 1: Skip this node (and entirely skip its
        subtree)
        long long skip = dp[jump][w];

        // Option 2: Take this node (if current capacity
        allows)
        long long take = 0;
        if (w >= weight[u]) {
            take = dp[i + 1][w - weight[u]] + value[u];
        }

        dp[i][w] = max(skip, take);
    }
}

// The answer is the state starting at pre-order index 1
with full capacity W
return dp[1][W];
}
};

/*
USAGE EXAMPLE:

signed main() {
    // Suppose we have 5 nodes (1 is the root)
    int n = 5;
    TreeKnapsack tk(n);

    // Values and weights for nodes 1 to 5
    tk.weight = {0, 1, 2, 3, 2, 1}; // 1-indexed (index 0 is
    unused)
    tk.value = {0, 5, 10, 12, 4, 3};

    // Tree structure: 1 is parent of 2 and 3. 2 is parent of
    4 and 5.
    // Directed edges from parent to child!
    tk.add_edge(1, 2);
    tk.add_edge(1, 3);
    tk.add_edge(2, 4);
    tk.add_edge(2, 5);

    int weight_limit = 4;

    // Calculate max value if we root at 1 and have max
    weight 4
    long long max_val = tk.solve(1, weight_limit);

    cout << "Max value within weight " << weight_limit << "
    is: " << max_val << "\n";
    // Output expected: node 1 (w=1, v=5) + node 2 (w=2,
    v=10) + node 5 (w=1, v=3)
    // Total weight = 4, Total value = 18.

    return 0;
}
*/

```

4 DP

4.1 DP Digit Optimization 1

```

/*
DIGIT DP OPTIMIZATION 1: REMOVING THE `is_less` (fr
DIMENSION
*/
vector<int> R;

```

```

pair<ll, ll> dp[20];

pair<ll, ll> calc(int pos, int fr) {
    if (pos == -1) return make_pair(0, 1);
    auto &res = dp[pos];
    if (fr && res != make_pair(-1LL, -1LL)) return res;
    int lim_r = (fr ? 9 : R[pos]);
    pair<ll, ll> val = make_pair(0, 0);
    for (int i = 0; i <= lim_r; i++) {
        pair<ll, ll> nxt = calc(pos - 1, fr | (i < lim_r));
        val.first += nxt.first + i * nxt.second;
        val.second += nxt.second;
    }
    return (fr ? res = val : val);
}

```

4.2 DP Digit Optimization 2

```

/*
    DIGIT DP OPTIMIZATION 2: ONE-TIME MEMSET FOR MULTI-TEST
    CASES
*/

const int mod = 1e9 + 7;

string R;
int dp[51][51][51][51];

int calc(int pos, int c3, int c6, int c9, int fr) {
    if (pos == -1) {
        return (c3 > 0 && c3 == c6 && c6 == c9);
    }
    auto &res = dp[pos][c3][c6][c9];
    if (fr && res != -1) return res;
    ll val = 0;
    int lim_r = (fr ? 9 : R[pos] - '0');
    for (int i = 0; i <= lim_r; i++) {
        val += calc(pos - 1, c3 + (i == 3), c6 + (i == 6), c9 + (i == 9), fr | (i < lim_r));
        if (val >= mod) val -= mod;
    }
    return (fr ? res = val : val);
}

int solve(string x) {
    reverse(x.begin(), x.end());
    R = x;
    return calc((int)R.size() - 1, 0, 0, 0, 0);
}

bool check(string x) {
    int c3 = 0, c6 = 0, c9 = 0;
    for (auto i : x) {
        c3 += (i == '3');
        c6 += (i == '6');
        c9 += (i == '9');
    }
    return (c3 > 0 && c3 == c6 && c6 == c9);
}

/*
// Usage in main():
// memset(dp, -1, sizeof dp); // Call this ONLY ONCE
// while (test--) {
//     string a, b; cin >> a >> b;
//     cout << (1LL * solve(b) - solve(a) + check(a) + mod) %
mod << "\n";
// }
*/

```

4.3 DP SoS

```

const int LOG = 20;
int dp[1 << LOG];

// 1. Sum over Submasks: dp[mask] = \sum_{sub \subseteq mask} A[sub]
// (sub is formed by turning OFF some bits of mask)
void sos_submask(int n, int a[]) {
    for (int mask = 0; mask < (1 << n); mask++) {

```

```

        dp[mask] = a[mask];
    }
    for (int i = 0; i < n; i++) {
        for (int mask = 0; mask < (1 << n); mask++) {
            if ((mask >> i) & 1) {
                dp[mask] += dp[mask ^ (1 << i)];
            }
        }
    }
}

// 2. Sum over Supermasks: dp[mask] = \sum_{mask \subseteq sup} A[sup]
// (sup is formed by turning ON some bits of mask)
void sos_supermask(int n, int a[]) {
    for (int mask = 0; mask < (1 << n); mask++) {
        dp[mask] = a[mask];
    }
    for (int i = 0; i < n; i++) {
        for (int mask = 0; mask < (1 << n); mask++) {
            if (!(mask >> i) & 1) {
                dp[mask] += dp[mask ^ (1 << i)];
            }
        }
    }
}

/*
COMMON USAGE PATTERNS & TRICKS:
1. Find pairs (u, v) such that u & v = 0:
2. Number of elements `x` such that (x & mask) == x:
3. Number of elements `x` such that (x & mask) == mask:
4. Finding two elements x, y with maximum (x | y):
*/

```

4.4 DP Divide And Conquer

```

/*
    DIVIDE AND CONQUER DP OPTIMIZATION

Idea:
- Optimizes DP equations of the form:
    dp[i][j] = min_{k < j} (dp[i-1][k] + cost(k+1, j))
- Condition: The optimal split point `opt(i, j)` must be
monotonic,
    meaning: opt(i, j) <= opt(i, j+1).
- This monotonicity allows us to use Divide & Conquer: if
we know the
    optimal split point for mid is `opt_mid`, then the split
point for
    any j < mid is <= opt_mid, and for any j > mid is >=
opt_mid.

Complexity:
- Original DP: O(K * N^2)
- D&C Optimized: O(K * N log N)

Memory Optimization:
- We only need the DP state from the previous phase (i -
1), so we can
    use two 1D arrays: `dp_before` and `dp_current`.
*/

const long long LINF = 1e18;

// DP arrays (1-indexed for the items)
vector<long long> dp_before, dp_current;

// 1. Problem-specific cost function
// Note: To keep the overall complexity O(K * N log N), this
cost function
// MUST be calculable in O(1). Usually done via prefix sums or
sliding window.
long long cost(int l, int r) {
    if (l > r) return 0;
    // TODO: Replace with the actual O(1) cost logic of the
problem
    return 0;
}

// 2. Divide and Conquer recursive function
// l, r: the range of `j` we are currently computing
dp_current[j] for.

```

```
// optl, optr: the valid range of the optimal split point `k`
// for this range.
void compute(int l, int r, int optl, int optr) {
    if (l > r) return;

    int mid = l + (r - l) / 2;

    // We want to find the optimal split point `k` for
    dp_current[mid]
    pair<long long, int> best = {LINF, -1};

    // k is the end of the previous segment, so the new segment
    // is [k+1, mid].
    // Note: k must be strictly less than `mid` if we require
    // non-empty segments,
    // or k <= mid depending on the problem formulation.
    for (int k = optl; k <= min(mid - 1, optr); k++) {
        // Check if the previous state is valid
        if (dp_before[k] != LINF) {
            long long current_val = dp_before[k] + cost(k + 1, mid);
            if (current_val < best.first) {
                best = {current_val, k};
            }
        }
    }

    dp_current[mid] = best.first;
    int opt = best.second;

    // 3. Recurse for the left and right halves using the
    // discovered monotonicity
    // For j < mid, optimal k is in [optl, opt]
    compute(l, mid - 1, optl, opt);

    // For j > mid, optimal k is in [opt, optr]
    compute(mid + 1, r, opt, optr);
}

/*
USAGE EXAMPLE:

signed main() {
    int n = 1000; // Number of elements
    int K = 10;   // Number of partitions

    dp_before.assign(n + 1, LINF);
    dp_current.assign(n + 1, LINF);

    // Base case for 0 partitions
    dp_before[0] = 0;
    // dp_before[x > 0] is LINF because we can't partition x
    // items into 0 parts

    for (int i = 1; i <= K; i++) {
        // Reset current DP array for the new partition step
        fill(dp_current.begin(), dp_current.end(), LINF);

        // Compute dp_current[1..n], knowing optimal k is in
        // [0..i-1]
        compute(1, n, 0, i - 1);

        // Move current row to before row for the next
        // iteration
        dp_before = dp_current;
    }

    cout << "Min cost to partition " << n << " items into "
    << K
        << " parts is: " << dp_current[n] << "\n";

    return 0;
}
*/
```

5 Strings

5.1 KMP

```
vector<int> compute_pi(const string& p) {
    int m = p.length();
```

```
vector<int> pi(m, 0);
for (int i = 1, j = 0; i < m; i++) {
    while (j > 0 && p[i] != p[j]) {
        j = pi[j - 1];
    }
    if (p[i] == p[j]) {
        j++;
    }
    pi[i] = j;
}
return pi;
}
```

5.2 Z Function

```
vector<int> z_function(const string& s) {
    int n = (int)s.length();
    vector<int> z(n, 0);
    for (int i = 1, l = 0, r = 0; i < n; i++) {
        if (i <= r) {
            z[i] = min(r - i + 1, z[i - l]);
        }
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
            z[i]++;
        }
        if (i + z[i] - 1 > r) {
            l = i;
            r = i + z[i] - 1;
        }
    }
    return z;
}
```

5.3 Suffix Array

```
/*
SUFFIX ARRAY & LCP ARRAY (O(N log N) Construction)
Idea:
- Suffix Array (sa): Sorts all suffixes of a string
  lexicographically.
  sa[i] stores the starting index of the i-th
  lexicographically
  smallest suffix.
- LCP Array (lcp): Stores the Longest Common Prefix between
  consecutive
  suffixes in the sorted Suffix Array. lcp[i] = LCP(sa[i],
  sa[i-1]).
Algorithm:
- Appends a special character '$' (strictly smaller than
  all other characters)
  to the end of the string.
- Sorts substrings of length 2^k using Radix Sort in O(N).
- Repeats log(N) times -> Total time O(N log N).
- Computes LCP in O(N) using Kasai's Algorithm.
*/

struct SuffixArray {
    int n;
    string s;
    vector<int> sa; // Suffix Array
    vector<int> c; // Equivalence classes
    vector<int> lcp; // Longest Common Prefix array

    // Constructor automatically builds SA and LCP
    SuffixArray(string const& _s) : s(_s + "$") {
        n = s.length();
        sa.assign(n, 0);
        c.assign(n, 0);
        lcp.assign(n, 0);

        build_sa();
        build_lcp();
    }

    void build_sa() {
```

```
const int alphabet = 256;
vector<int> cnt(max(alphabet, n), 0);
vector<int> p(n), pn(n), cn(n);

// 1. Initial sort for length 1 (k = 0)
```



```

for (int i = 0; i < n; i++) cnt[s[i]]++;
for (int i = 1; i < alphabet; i++) cnt[i] += cnt[i - 1];
for (int i = 0; i < n; i++) p[--cnt[s[i]]] = i;

c[p[0]] = 0;
int classes = 1;
for (int i = 1; i < n; i++) {
    if (s[p[i]] != s[p[i - 1]]) classes++;
    c[p[i]] = classes - 1;
}

// 2. Transitions for length 2^h
for (int h = 0; (1 << h) < n; h++) {
    // Shift by 2^h
    for (int i = 0; i < n; i++) {
        pn[i] = p[i] - (1 << h);
        if (pn[i] < 0) pn[i] += n;
    }

    // Radix Sort on the first half
    fill(cnt.begin(), cnt.begin() + classes, 0);
    for (int i = 0; i < n; i++) cnt[c[pn[i]]]++;
    for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
    for (int i = n - 1; i >= 0; i--) p[--cnt[c[pn[i]]]] = pn[i];

    // Update equivalence classes
    cn[p[0]] = 0;
    classes = 1;
    for (int i = 1; i < n; i++) {
        pair<int, int> cur = {c[p[i]], c[(p[i] + (1 << h)) % n]};
        pair<int, int> prev = {c[p[i - 1]], c[(p[i - 1] + (1 << h)) % n]};
        if (cur != prev) classes++;
        cn[p[i]] = classes - 1;
    }
    c.swap(cn);
}
sa = p;
}

// Kasai's algorithm to build LCP array in O(N)
void build_lcp() {
    vector<int> rank(n, 0);
    for (int i = 0; i < n; i++) {
        rank[sa[i]] = i;
    }

    int k = 0;
    // Iterate through original string indices (excluding the
    // appended '$')
    for (int i = 0; i < n - 1; i++) {
        int pi = rank[i];
        int j = sa[pi - 1]; // The suffix preceding the current
        // one in SA

        // Expand the matching prefix
        while (i + k < n && j + k < n && s[i + k] == s[j + k]) {
            k++;
        }

        lcp[pi] = k;
        if (k > 0) k--; // Move to the next suffix, LCP
        // decreases by at most 1
    }
}

/*
USAGE EXAMPLES:

signed main() {
    string str = "ababba";
    SuffixArray sa(str);

    // 1. Number of Distinct Substrings
    // Formula: Total substrings - sum of LCP array
    // Total substrings of length N is N * (N + 1) / 2
    long long N = str.length();
    long long distinct_substrings = N * (N + 1) / 2;

```

```

for (int i = 1; i <= N; i++) {
    distinct_substrings -= sa.lcp[i];
}
cout << "Distinct substrings: " << distinct_substrings <<
"\n";

// 2. Printing Suffix Array & LCP Array
cout << "i\tsA[i]\tLCP[i]\tSuffix\n";
// Note: sa.sa[0] is always the appended '$' (length 0
// suffix)
for (int i = 1; i <= N; i++) {
    cout << i << "\t"
        << sa.sa[i] << "\t"
        << sa.lcp[i] << "\t"
        << str.substr(sa.sa[i]) << "\n";
}

return 0;
}
*/

```

5.4 Aho Corasick

```

/*
AHO-CORASICK AUTOMATON
Idea:
- Multi-pattern string search algorithm.
- Constructs a Trie from a set of patterns and adds:
    1. Failure Links (`fail`): Where to go when a character
    mismatch occurs.
    2. Output Links (`output`): Direct link to the nearest
    valid pattern
        ending in the failure chain (handles
        overlapping/substring patterns).
- OPTIMIZATION: We convert the Trie into a Deterministic
Finite Automaton
(DFA). Transition `next[c]` either points to the actual
child OR the
fallback state in O(1).

Complexity:
- Build Time: O(Sum of pattern lengths * Alphabet Size)
- Search Time: O(Length of Text + Number of Matches)
*/

const int ALPHABET = 26;
const char BASE_CHAR = 'a'; // Change to 'A' or '0' depending
on the problem

struct ACNode {
    int next[ALPHABET];
    int fail;
    int output; // Also known as dict_link (points to the
    // nearest terminal node)
    vector<int> pattern_ids; // IDs of patterns that end exactly
    // at this node

    ACNode() {
        fill(begin(next), end(next), 0);
        fail = 0;
        output = 0;
    }
};

struct AhoCorasick {
    vector<ACNode> t;

    AhoCorasick() {
        t.emplace_back(); // Root node at index 0
    }

    // 1. Insert a pattern into the Trie
    void insert(const string& s, int id) {
        int cur = 0;
        for (char c : s) {
            int v = c - BASE_CHAR;
            if (!t[cur].next[v]) {
                t[cur].next[v] = t.size();
                t.emplace_back();
            }
            cur = t[cur].next[v];

```

```

    }
    t[cur].pattern_ids.push_back(id);
}

// 2. Build Failure and Output links via BFS
void build() {
    queue<int> q;

    // Initialize root's children
    for (int i = 0; i < ALPHABET; i++) {
        if (t[0].next[i]) {
            t[t[0].next[i]].fail = 0;
            q.push(t[0].next[i]);
        }
    }

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int i = 0; i < ALPHABET; i++) {
            int v = t[u].next[i];
            if (v) {
                // If child exists, set its failure link to the
                // child of u's fail node
                t[v].fail = t[t[u].fail].next[i];

                // Optimization: Output link points to the nearest
                // pattern end
                if (!t[t[v].fail].pattern_ids.empty()) {
                    t[v].output = t[t[v].fail].output;
                } else {
                    t[v].output = t[t[v].fail].output;
                }

                q.push(v);
            } else {
                // Trie Automaton Optimization: If child doesn't
                // exist,
                // point directly to the fallback state. This makes
                // text traversal O(1).
                t[u].next[i] = t[t[u].fail].next[i];
            }
        }
    }
}

// 3. Search for patterns in the given text
void search(const string& text) {
    int cur = 0;
    for (int i = 0; i < text.length(); i++) {
        int c = text[i] - BASE_CHAR;
        cur = t[cur].next[c]; // O(1) transition due to
        // Automaton optimization

        // Traverse output links to find all patterns ending at
        // current position i
        int temp = cur;
        while (temp != 0) {
            for (int id : t[temp].pattern_ids) {
                // You can store or process the match here
                cout << "Pattern ID " << id << " ends at index " <<
                i << "\n";
            }
            // Jump to the next valid pattern in the failure chain
            // If the current node doesn't have an end word but
            // has an output link,
            // this safely jumps to it.
            if (t[temp].pattern_ids.empty()) {
                temp = t[temp].output;
            } else {
                temp = t[temp].output;
            }
        }
    }
}

/*
USAGE EXAMPLE:

```

```

signed main() {
    AhoCorasick ac;

    vector<string> patterns = {"he", "she", "his", "hers"};
    for (int i = 0; i < patterns.size(); i++) {
        ac.insert(patterns[i], i);
    }

    // MUST call build() after inserting all patterns and
    // before searching
    ac.build();

    string text = "ushers";
    cout << "Searching in text: " << text << "\n";
    ac.search(text);

    // Output expected:
    // Pattern ID 1 ("she") ends at index 3
    // Pattern ID 0 ("he") ends at index 3
    // Pattern ID 3 ("hers") ends at index 5

    return 0;
}
*/

```

6 Math

6.1 Extended Euclid

```

// 1. Core Extended GCD
// Returns gcd(a, b) and updates x, y via reference.
long long extgcd(long long a, long long b, long long &x, long
long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long d = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

// 2. Solve Linear Diophantine Equation: ax + by = c
// Returns true if a solution exists, false otherwise.
// If true, (x0, y0) stores one valid base solution, and g
stores gcd(a, b).
//
// GENERAL SOLUTION GENERATOR:
// All solutions can be generated using an integer k:
// x = x0 + k * (b / g)
// y = y0 - k * (a / g)
bool solve_diophantine(long long a, long long b, long long c,
long long &x0, long long &y0, long long &g) {
    // Solve for abs(a) and abs(b) first to maintain correct
    // signs
    g = extgcd(abs(a), abs(b), x0, y0);

    if (c % g != 0) {
        return false; // No solution if c is not divisible by
        gcd(a, b)
    }

    x0 *= c / g;
    y0 *= c / g;

    // Adjust signs back
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;

    return true;
}

// 3. Modular Multiplicative Inverse
// Solves: ax = 1 (mod m).
// Valid ONLY IF a and m are coprime (gcd(a, m) == 1).
// Returns -1 if the inverse does not exist.
long long mod_inverse(long long a, long long m) {
    long long x, y;

```

```

long long g = extgcd(a, m, x, y);

if (g != 1) {
    return -1; // Inverse doesn't exist
}

// x can be negative, so we adjust it to be in the range [0, m - 1]
return (x % m + m) % m;
}

```

6.2 CRT

```

/*
GENERALIZED CHINESE REMAINDER THEOREM (CRT)
Idea:
- Solves a system of linear congruences:
    x = a_1 (mod m_1)
    x = a_2 (mod m_2)
    ...
    x = a_n (mod m_n)
- Unlike the standard CRT, this GENERALIZED version does NOT require the moduli (m_i) to be pairwise coprime.
- It works by iteratively merging two congruences into one.
Math:
- Merging x = a1 (mod m1) and x = a2 (mod m2):
    We need x = a1 + k1 * m1 = a2 + k2 * m2
    => k1 * m1 - k2 * m2 = a2 - a1
    This is a linear Diophantine equation. Let g = gcd(m1, m2).
    If (a2 - a1) is not divisible by g, there is NO SOLUTION.
    Otherwise, we solve for k1 and find the new merged congruence:
    x = new_a (mod lcm(m1, m2)).
Complexity: O(N log(LCM))
Note: Uses __int128_t to prevent overflow during intermediate multiplications.
*/
long long extgcd(long long a, long long b, long long &x, long long &y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    long long x1, y1;
    long long d = extgcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

// Generalized CRT
// Inputs: vector of remainders `a`, vector of moduli `m`.
// Returns: A pair {ans_a, ans_m} such that x = ans_a (mod ans_m).
// If no solution exists, returns {-1, -1}.
pair<long long, long long> crt(const vector<long long>& a, const vector<long long>& m) {
    assert(a.size() == m.size());
    if (a.empty()) return {0, 1}; // Trivial case

    long long ans_a = a[0];
    long long ans_m = m[0];

    for (int i = 1; i < (int)a.size(); i++) {
        long long cur_a = a[i];
        long long cur_m = m[i];

        long long u, v;
        long long g = extgcd(ans_m, cur_m, u, v);

        // Check if a solution exists for this pair
        if ((cur_a - ans_a) % g != 0) {
            return {-1, -1}; // No solution
        }

        long long lcm = (ans_m / g) * cur_m;

        // Calculate k1 = (a2 - a1) / g * u (mod m2 / g)

```

```

// Using __int128_t to avoid overflow when multiplying
// (cur_a - ans_a) / g and u
long long diff = cur_a - ans_a;
__int128_t k = (__int128_t)diff / g * u;
k %= (cur_m / g);

// Substitute k back to find the new remainder
ans_a = ans_a + (long long)k * ans_m;

// Normalize ans_a to be positive and within [0, lcm - 1]
ans_a = (ans_a % lcm + lcm) % lcm;
ans_m = lcm;
}

return {ans_a, ans_m};
}

/*
USAGE EXAMPLE:

signed main() {
    // System of equations:
    // x = 2 (mod 3)
    // x = 3 (mod 5)
    // x = 2 (mod 7)

    vector<long long> a = {2, 3, 2};
    vector<long long> m = {3, 5, 7};

    pair<long long, long long> res = crt(a, m);

    if (res.first == -1) {
        cout << "No solution exists.\n";
    } else {
        cout << "Smallest positive solution: x = " << res.first << "\n";
        cout << "General solution: x = " << res.first << " (mod " << res.second << ")\n";
    }

    // Output:
    // Smallest positive solution: x = 23
    // General solution: x = 23 (mod 105)

    return 0;
}
*/

```

6.3 Lucas Theorem

```

/*
LUCAS THEOREM
Idea:
- Computes nCr(n, k) % p where n and k can be very large (up to 10^18), but p is a relatively small PRIME number (e.g., p <= 10^6).
- Theorem: nCr(n, k) = Product of nCr(n_i, k_i) (mod p), where n_i and k_i are the digits of n and k in base p.
- If at any point k_i > n_i, then nCr(n_i, k_i) = 0, meaning the entire result becomes 0.
Complexity:
- Precomputation: O(p) to compute factorials up to p - 1.
- Query: O(log_p(n)) per nCr query.
- Space: O(p)
Requirement:
- p MUST be a prime number.
*/
struct LucasTheorem {
    long long p;
    vector<long long> fact, invFact;

    long long power(long long base, long long exp) {
        long long res = 1;
        base %= p;
        while (exp > 0) {
            if (exp % 2 == 1) res = (res * base) % p;
            base = (base * base) % p;
            exp /= 2;
        }
    }
};

```

```

    }
    return res;
}

long long modInverse(long long n) {
    return power(n, p - 2);
}

// Constructor: Precomputes factorials and inverse
// factorials up to p - 1
LucasTheorem(long long prime_mod) {
    p = prime_mod;
    fact.assign(p, 1);
    invFact.assign(p, 1);

    for (int i = 1; i < p; i++) {
        fact[i] = (fact[i - 1] * i) % p;
    }

    invFact[p - 1] = modInverse(fact[p - 1]);
    for (int i = p - 2; i >= 0; i--) {
        invFact[i] = (invFact[i + 1] * (i + 1)) % p;
    }
}

// Computes nCr(n, k) % p for small values (n, k < p)
long long nCr_small(long long n, long long k) {
    if (k < 0 || k > n) return 0;
    long long num = fact[n];
    long long den = (invFact[k] * invFact[n - k]) % p;
    return (num * den) % p;
}

// Computes nCr(n, k) % p for large values using Lucas
// Theorem
long long lucas(long long n, long long k) {
    if (k < 0 || k > n) return 0;

    long long res = 1;
    while (n > 0 || k > 0) {
        long long ni = n % p;
        long long ki = k % p;

        if (ki > ni) return 0;

        res = (res * nCr_small(ni, ki)) % p;

        n /= p;
        k /= p;
    }
    return res;
}
};

```

6.4 Euler Totient

```

/*
EULER'S TOTIENT FUNCTION (PHI FUNCTION)
Idea:
- phi(n) counts the number of integers between 1 and n
inclusive
that are coprime to n (i.e., gcd(x, n) == 1).
Important Properties for CP:
1. If p is prime: phi(p) = p - 1
2. If p is prime and k >= 1: phi(p^k) = p^k - p^(k-1)
3. Multiplicative: If gcd(a, b) == 1, then phi(a * b) =
phi(a) * phi(b)
4. Euler's Theorem: If gcd(a, m) == 1, then a^phi(m) = 1
(mod m)
5. Sum of phi over divisors: The sum of phi(d) for all
divisors d of n
is exactly equal to n. (Sum_{d|n} phi(d) = n)
*/
// 1. Single Computation
// Computes phi(n) in O(sqrt(n)) time.
// Uses the formula: phi(n) = n * (1 - 1/p1) * (1 - 1/p2) *
...
// where p_i are the distinct prime factors of n.
long long get_phi(long long n) {
    long long result = n;
    for (long long i = 2; i * i <= n; i++) {

```

```

        if (n % i == 0) {
            while (n % i == 0) {
                n /= i;
            }
            result -= result / i;
        }
    }
    if (n > 1) {
        result -= result / n;
    }
    return result;
}

// 2. Sieve Computation (Batch Computation)
// Computes phi(i) for all 1 <= i <= N in O(N log log N) time.
// Similar to the Sieve of Eratosthenes.
vector<int> phi_sieve(int n) {
    vector<int> phi(n + 1);

    for (int i = 0; i <= n; i++) {
        phi[i] = i;
    }

    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i) {
                phi[j] -= phi[j] / i;
            }
        }
    }
    return phi;
}

/*
USAGE EXAMPLE:
signed main() {
    // 1. Single query for a large number
    long long n = 12;
    cout << "phi(" << n << ") = " << get_phi(n) << "\n";
    // Output: 4 (Coprimes are 1, 5, 7, 11)
    // 2. Precomputing phi for all numbers up to MAX_N
    int MAX_N = 100000;
    vector<int> phi = phi_sieve(MAX_N);
    cout << "phi(10) = " << phi[10] << "\n"; // Output: 4
    cout << "phi(97) = " << phi[97] << "\n"; // Output: 96
    (Since 97 is prime)
    return 0;
}
*/

```

6.5 Mobius Function

```

/*
MOBIUS FUNCTION & INVERSION

Definition of mu(n):
- mu(n) = 1 if n = 1
- mu(n) = 0 if n has a squared prime factor (e.g., 4, 9,
12, 18)
- mu(n) = 1 if n is square-free with an EVEN number of
prime factors
- mu(n) = -1 if n is square-free with an ODD number of
prime factors

Important Properties for CP:
1. Sum over divisors: The sum of mu(d) for all divisors d
of n is 1
if n = 1, and 0 otherwise. (Sum_{d|n} mu(d) = [n == 1])
-> This is the most used property to replace [gcd(i, j)
== 1].
2. Mobius Inversion Formula:
If g(n) = Sum_{d|n} f(d), then f(n) = Sum_{d|n} mu(d) *
g(n/d).
*/
// 1. Single Computation
// Computes mu(n) in O(sqrt(n)) time.
int get_mu(long long n) {
    if (n == 1) return 1;
    int prime_count = 0;

    for (long long i = 2; i * i <= n; i++) {

```

```

    if (n % i == 0) {
        int count = 0;
        while (n % i == 0) {
            count++;
            n /= i;
        }
        if (count > 1) return 0;
        prime_count++;
    }
}

if (n > 1) {
    prime_count++;
}

return (prime_count % 2 == 1) ? -1 : 1;
}

// 2. Linear Sieve Computation (Batch Computation)
// Computes mu(i) for all 1 <= i <= N in exactly O(N) time.
struct MobiusSieve {
    int n;
    vector<int> mu;
    vector<int> primes;
    vector<bool> is_prime;

    MobiusSieve(int _n) {
        n = _n;
        mu.assign(n + 1, 0);
        is_prime.assign(n + 1, true);

        is_prime[0] = is_prime[1] = false;
        mu[1] = 1;

        for (int i = 2; i <= n; i++) {
            if (is_prime[i]) {
                primes.push_back(i);
                mu[i] = -1; // Prime numbers have exactly 1 prime
                           // factor (ODD)
            }
            for (int p : primes) {
                if (i * p > n) break;
                is_prime[i * p] = false;

                if (i % p == 0) {
                    // i * p is divisible by p^2
                    mu[i * p] = 0;
                    break;
                } else {
                    // p is a new distinct prime factor
                    mu[i * p] = -mu[i];
                }
            }
        }
    }
};

/*
USAGE EXAMPLE:

signed main() {
    // 1. Single query for a large number
    long long n = 30; // 30 = 2 * 3 * 5 (3 prime factors ->
    odd -> mu = -1)
    cout << "mu(" << n << ") = " << get_mu(n) << "\n"; //
    Output: -1

    long long m = 12; // 12 = 2^2 * 3 (has squared prime
    factor 2^2 -> mu = 0)
    cout << "mu(" << m << ") = " << get_mu(m) << "\n"; //
    Output: 0

    // 2. Precomputing mu for all numbers up to MAX_N in O(N)
    int MAX_N = 100000;
    MobiusSieve sieve(MAX_N);

    cout << "mu(10) = " << sieve.mu[10] << "\n"; // 10 = 2 *
    5 -> Output: 1
    cout << "mu(97) = " << sieve.mu[97] << "\n"; // 97 is
    prime -> Output: -1

    return 0;
}

```

```

    }
}
*/

```

6.6 Miller Rabin

```

/*
MILLER-RABIN PRIMALITY TEST

Idea:
- A fast primality test. While originally probabilistic, it
  can be made
  DETERMINISTIC for all 64-bit integers by testing a
  specific, small set
  of prime bases.
- To test if n is prime, we write n - 1 = d * 2^s, where d
  is odd.
- For a base 'a', n is composite if a^d != 1 (mod n) AND
  a^(d * 2^r) != n - 1 (mod n)
  for all r in [0, s - 1].

Bases for Deterministic Test:
- For n < 2^64, testing the first 12 primes is strictly
  sufficient:
  {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37}

Complexity:
- Time: O(K log n) where K is the number of bases (K <=
  12).
- Space: O(1)
- Uses __uint128_t to cleanly prevent overflow during
  64-bit multiplications.
*/
using u64 = uint64_t;
using u128 = __uint128_t;

// Modular multiplication using 128-bit integer to avoid
// overflow
u64 mod_mul(u64 a, u64 b, u64 m) {
    return (u64)((u128)a * b % m);
}

// Modular exponentiation: (base^exp) % mod
u64 mod_pow(u64 base, u64 exp, u64 mod) {
    u64 res = 1;
    base %= mod;
    while (exp > 0) {
        if (exp % 2 == 1) res = mod_mul(res, base, mod);
        base = mod_mul(base, base, mod);
        exp /= 2;
    }
    return res;
}

// Miller-Rabin deterministic primality test for 64-bit
// integers
bool is_prime(u64 n) {
    if (n < 2) return false;
    if (n == 2 || n == 3) return true;
    if (n % 2 == 0) return false;

    // Write n - 1 as d * 2^s
    u64 d = n - 1;
    int s = 0;
    while (d % 2 == 0) {
        d /= 2;
        s++;
    }

    // Deterministic bases for values up to 2^64
    static const u64 bases[] = {2, 3, 5, 7, 11, 13, 17, 19, 23,
    29, 31, 37};

    for (u64 a : bases) {
        // If n is smaller than or equal to the base, we are done
        // (since n is prime and has already passed the n % 2 == 0
        // check,
        // it would just be one of the prime bases).
        if (n <= a) break;

        u64 x = mod_pow(a, d, n);
        if (x == 1 || x == n - 1) continue;
    }
}

```

```

bool composite = true;
for (int r = 1; r < s; r++) {
    x = mod_mul(x, x, n);
    if (x == n - 1) {
        composite = false;
        break;
    }
}

if (composite) return false;
}

return true;
}

/*
USAGE EXAMPLE:

signed main() {
    long long n = 100000000000000003LL; // A large 64-bit
    number

    if (is_prime(n)) {
        cout << n << " is PRIME.\n";
    } else {
        cout << n << " is COMPOSITE.\n";
    }

    // Expected Output: 100000000000000003 is PRIME.

    return 0;
}
*/

```

7 Geometry

8 Miscellaneous

8.1 Alien Trick

```

/*
ALIEN'S TRICK (WQS BINARY SEARCH / LAGRANGE RELAXATION)

Idea:
- Solves DP problems with an "exactly K items/partitions"
  constraint.
- We drop the K constraint and add a penalty `lambda` for
  each partition.
- We binary search on `lambda`. If we use > K partitions,
  we increase
  the penalty. If we use < K partitions, we decrease it.
- To handle collinear points on the convex hull, we always
  prefer the
  MAXIMUM (or MINIMUM) number of partitions for the same
  cost, and use
  `cnt >= K` in the binary search.

Condition:
- Let f(x) be the optimal cost using exactly x partitions.
- f(x) MUST be convex or concave.

Complexity: O(N * log(MAX_PENALTY)) instead of O(N * K).
*/
const long long LINP = 1e18;

int n;
vector<long long> a; // Input array

// 1. The 1D DP function without the 'K' constraint
// Returns a pair: {optimal_cost_with_penalty,
number_of_partitions_used}
pair<long long, int> solve_dp(long long lambda) {
    // dp[i] stores {min_cost, max_partitions_used} up to index
    i
    vector<pair<long long, int>> dp(n + 1, {LINP, 0});
    dp[0] = {0, 0};

    for (int i = 1; i <= n; i++) {
        // Iterate over all possible previous split points

```

```

// Note: In real problems, this inner loop is often
optimized to O(1)
// using Convex Hull Trick, D&C, or Monotone Queue.
for (int j = 0; j < i; j++) {
    // cost(j+1, i) is the cost of the segment from j+1 to i
    // We ADD `lambda` as a penalty for creating this new
    segment
    long long segment_cost = 0; /* TODO: Calculate real cost
    */
    long long new_cost = dp[j].first + segment_cost +
    lambda;
    int new_cnt = dp[j].second + 1;

    // We want to MINIMIZE cost.
    // TIE-BREAKER: If costs are equal, MAXIMIZE the number
    of partitions.
    // This tie-breaker is CRUCIAL to handle collinear
    points on the convex hull!
    if (new_cost < dp[i].first || (new_cost == dp[i].first
    && new_cnt > dp[i].second)) {
        dp[i] = {new_cost, new_cnt};
    }
}
}
return dp[n];
}

/*
USAGE EXAMPLE:

signed main() {
    n = 1000;
    int K = 50; // We need exactly K partitions

    // Binary search for the optimal penalty `lambda`
    // The range depends on the maximum possible cost
    difference
    long long l = -1e12, r = 1e12;
    long long ans = -1;

    while (l <= r) {
        long long mid = l + (r - l) / 2;

        pair<long long, int> res = solve_dp(mid);
        long long cost = res.first;
        int cnt = res.second;

        // If we used >= K partitions, it means the penalty is
        small enough
        // to allow at least K partitions. We record the answer
        and try to
        // INCREASE the penalty to force fewer partitions.
        if (cnt >= K) {
            // The real cost is the DP cost MINUS the penalty we
            added for K partitions
            ans = cost - K * mid;
            l = mid + 1; // Increase penalty
        } else {
            r = mid - 1; // Decrease penalty
        }
    }

    cout << "Minimum cost using exactly " << K << "
    partitions: " << ans << "\n";

    return 0;
}
*/

```

8.2 Interval Container

```

/*
INTERVAL CONTAINER (DISJOINT INTERVALS SET)
Idea:
- Maintains a set of disjoint intervals [L, R].
- Automatically merges overlapping or adjacent intervals
  when adding.
- Automatically splits intervals when removing a segment.
- Dynamically maintains the total length/count of covered
  points.
Complexity:

```

```

- add_interval: Amortized  $O(\log N)$ 
- remove_interval: Amortized  $O(\log N)$ 
- contains:  $O(\log N)$ 
*/
const long long LINF = 2e18;

struct IntervalContainer {
    // Stores intervals as [L, R].
    // R is inclusive. For example, [1, 3] covers points 1, 2,
    // and 3.
    set<pair<long long, long long>> st;
    long long total_length = 0;

    // 1. Add an interval [L, R] and merge with
    // overlapping/adjacent ones
    void add_interval(long long L, long long R) {
        if (L > R) return;

        // Find the first interval that ends at or after L - 1
        // (We use L - 1 because we want to merge adjacent
        // intervals like [1, 2] and [3, 4])
        auto it = st.lower_bound({L - 1, -LINF});
        if (it != st.begin()) {
            auto prev = it;
            --prev;
            if (prev->second >= L - 1) {
                it = prev;
            }
        }

        long long nl = L, nr = R;
        // Iterate and erase all intervals that overlap with [L,
        // R]
        // (We use R + 1 to merge adjacent intervals)
        while (it != st.end() && it->first <= R + 1) {
            nl = min(nl, it->first);
            nr = max(nr, it->second);
            total_length -= (it->second - it->first + 1);
            it = st.erase(it); // C++11: erase returns the iterator
            // to the next element
        }

        st.insert({nl, nr});
        total_length += (nr - nl + 1);
    }

    // 2. Remove an interval [L, R] (punch a hole)
    void remove_interval(long long L, long long R) {
        if (L > R) return;

        // Find the first interval that ends at or after L
        auto it = st.lower_bound({L, -LINF});
        if (it != st.begin()) {
            auto prev = it;
            --prev;
            if (prev->second >= L) {
                it = prev;
            }
        }

        // Process all intervals that overlap with [L, R]
        while (it != st.end() && it->first <= R) {
            long long il = it->first;
            long long ir = it->second;
            total_length -= (ir - il + 1);
            it = st.erase(it);

            // If the original interval extends to the left of the
            // removed segment, keep that part
            if (il < L) {
                st.insert({il, L - 1});
                total_length += (L - 1 - il + 1);
            }

            // If the original interval extends to the right of the
            // removed segment, keep that part
            // Note: The new right part has a left bound > R, so it
            // won't be processed again by the while loop.
            if (ir > R) {
                st.insert({R + 1, ir});
                total_length += (ir - (R + 1) + 1);
            }
        }
    }
};

```

```

    }
}

// 3. Check if a specific point x is covered by any interval
bool contains(long long x) {
    // upper_bound finds the first interval whose L is
    // strictly greater than x
    auto it = st.upper_bound({x, LINF});
    if (it == st.begin()) return false;
    --it; // The interval before it has L <= x
    return it->second >= x;
}

// 4. Get total covered points length
long long get_length() {
    return total_length;
}

/*
USAGE EXAMPLE:
signed main() {
    IntervalContainer ic;

    ic.add_interval(1, 5);
    ic.add_interval(7, 10);
    cout << "Length: " << ic.get_length() << "\n"; // (5-1+1)
    // + (10-7+1) = 5 + 4 = 9

    // Merge adjacent and overlapping segments
    // Adding [4, 8] will merge [1, 5] and [7, 10] into [1,
    // 10]
    ic.add_interval(4, 8);
    cout << "Length: " << ic.get_length() << "\n"; // Output:
    // 10

    // Punch a hole in the middle
    ic.remove_interval(3, 6);
    cout << "Length: " << ic.get_length() << "\n"; //
    // Remainder: [1, 2] and [7, 10] -> Length: 6

    // Check points
    cout << "Contains 2? " << ic.contains(2) << "\n"; //
    // Output: 1 (true)
    cout << "Contains 5? " << ic.contains(5) << "\n"; //
    // Output: 0 (false)

    return 0;
}
*/

```

8.3 Ternary Search

```

/*
Idea:
- Finds the minimum or maximum of a Unimodal function (a
  function that
  strictly increases then strictly decreases, or
  vice-versa).
Two variants:
1. Floating-point: Iterate a fixed number of times (e.g.,
  200) instead
  of testing  $(R - L > \epsilon)$ . 200 iterations shrink the
  search space by
   $(2/3)^{200}$ , which is enough for any floating-point
  precision limit.
2. Integer: Search until the range [L, R] is small (e.g.,
  length <= 3),
  then use a simple linear scan. This avoids infinite
  loops caused by
  integer division rounding.

/*
// The unimodal function we want to optimize.
// Example:  $f(x) = (x - 5)^2 + 10$  (Minimum is at  $x = 5$ )
double f_double(double x) {
    return (x - 5.0) * (x - 5.0) + 10.0;
}

long long f_int(long long x) {
    return (x - 5) * (x - 5) + 10;
}

```

```

}

struct TernarySearch {
    // 1. Ternary Search on Real Numbers (Floating-point)
    // Returns the x-coordinate of the MINIMUM.
    // To find MAXIMUM: change `if (f_double(m1) <
    f_double(m2))` to `>`
    double solve_double(double L, double R) {
        // 200 iterations are ALWAYS enough to achieve maximum
        double precision
        for (int i = 0; i < 200; i++) {
            double m1 = L + (R - L) / 3.0;
            double m2 = R - (R - L) / 3.0;

            if (f_double(m1) < f_double(m2)) {
                R = m2; // The minimum is closer to m1, discard [m2,
                R]
            } else {
                L = m1; // The minimum is closer to m2, discard [L,
                m1]
            }
        }
        return L; // L and R are practically identical now
    }

    // 2. Ternary Search on Integers
    // Returns the x-coordinate of the MINIMUM.
    // To find MAXIMUM: change `<` to `>` in BOTH the ternary
    condition
    // and the final linear scan.
    long long solve_int(long long L, long long R) {
        // Stop when the range is small enough (3 points or fewer)
        while (R - L > 2) {
            long long m1 = L + (R - L) / 3;
            long long m2 = R - (R - L) / 3;

            if (f_int(m1) < f_int(m2)) {
                R = m2;
            } else {
                L = m1;
            }
        }

        // Linear scan for the remaining elements to avoid integer
        rounding bugs
        long long best_x = L;
        long long best_val = f_int(L);
        for (long long i = L + 1; i <= R; i++) {
            long long current_val = f_int(i);
            if (current_val < best_val) {
                best_val = current_val;
                best_x = i;
            }
        }

        return best_x;
    }
};

/*
USAGE EXAMPLE:
signed main() {
    TernarySearch ts;

    // Floating point search in range [-1000.0, 1000.0]
    double ans_d = ts.solve_double(-1000.0, 1000.0);
    cout << fixed << setprecision(6);
    cout << "Min (Double) at x = " << ans_d << ", f(x) = " <<
    f_double(ans_d) << "\n";

    // Integer search in range [-1000, 1000]
    long long ans_i = ts.solve_int(-1000, 1000);
    cout << "Min (Int) at x = " << ans_i << ", f(x) = " <<
    f_int(ans_i) << "\n";

    // Expected output for both:
    // x = 5, f(x) = 10

    return 0;
}
*/

```